Neurocomputing 124 (2014) 162–167

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Optimizing urban traffic flow using Genetic Algorithm with Petri net analysis as fitness function

Henrique Dezani *, Regiane D.S. Bassi, Norian Marranghello, Luís Gomes, Furio Damiani, Ivan Nunes da Silva

Faculty of Technology of Rio Preto (FATEC Rio Preto), Rua Fernandópolis, 2510, Eldorado, São José do Rio Preto, São Paulo, Brazil

ARTICLE INFO

Article history:
Received 5 November 2012
Received in revised form
20 July 2013
Accepted 30 July 2013
Communicated by R. Capobianco Guido
Available online 24 August 2013

Keywords:
Urban traffic
Genetic Algorithm
Petri net
Optimization

ABSTRACT

This paper describes a new methodology adopted for urban traffic stream optimization. By using Petri net analysis as fitness function of a Genetic Algorithm, an entire urban road network is controlled in real time. With the advent of new technologies that have been published, particularly focusing on communications among vehicles and roads infrastructures, we consider that vehicles can provide their positions and their destinations to a central server so that it is able to calculate the best route for one of them. Our tests concentrate on comparisons between the proposed approach and other algorithms that are currently used for the same purpose, being possible to conclude that our algorithm optimizes traffic in a relevant manner.

1. Introduction

With the increasing number of vehicles traveling on urban roads and the impossibility, in some cases, to restructure these roads to support new demands, some traffic congestion is generated, directly affecting people's quality of life as well as the logistics of companies, industries and public services. Given this problem, there is a need to create control systems that aim to alleviate urban traffic situation. Navigation systems embedded in vehicles compute the most appropriate route based on the current position of the vehicle, the driver's desired destination, and parameters such as the shortest possible distance or the minimum travel time. Some of these systems utilize information provided by traffic controllers or Internet in order to avoid some routes that may be congested. However, these systems define the routes based on the decision of only one driver. While providing alternate routes in an attempt to avoid jammed places, they can possibly congest other routes as they do not consider the routes of other drivers.

For the system to be collaborative in such a way that vehicles can know the route of other vehicles and then define optimal routes for everyone, it is necessary that the information of the current location and destiny of these vehicles are reported and analyzed by a global system. The exchange of information between Vehicles and Infrastructure (V2I) has been widely investigated and interesting results have already been achieved [1,2]. The aid in defining the path to be followed by the driver is part of the Advanced Traveller Information System (ATIS), which makes use of emerging technologies such as Internet, mobile devices and wireless communication to define such paths based on information from the environment [3,4]. The ATIS is part of Intelligent Transport Systems (ITS), whose focus is on sustainability and integration of systems already deployed and new systems for coordinating transportation systems safely and efficiently, performing the monitoring, controlling and provisioning of information relevant to traffic [5].

Since the behavior of urban traffic stream is asynchronous and parallel as regards to intersections and traffic lights, Petri nets (PNs) [6] have become relevant to the project for their ability to analyze control systems, as they present such characteristics. With PNs one can scan the state space and simulate a given network [7]. These analyzes identify, e.g., the reachability of markings, deadlocks and conflicts between transitions, useful in identifying jammed places over time based on the places flown by the markings.

There are several works that try to find the best time of each interval so as to avoid congestion in the queues. Determining the time-intervals of traffic signals based on the number of vehicles on roads, without routing information, is difficult. Thus, it is not easy to reduce the waiting-time for all vehicles that follow the same route provided by GPS systems. In general, GPS systems use some algorithm to define the best route based on time or distance of

* Corresponding author. Tel.: +55 17 99135 4070.
E-mail addresses: dezani@fatecriopreto.edu.br, dezani@dsif.fee.unicamp.br (H. Dezani), regiane.solgon@usp.br (R.D.S. Bassi), norian@ibilce.unesp.br (N. Marranghello), lugo@fct.unl.pt (L. Gomes), furio@dsif.fee.unicamp.br (F. Damiani), insilva@sc.usp.br (I. Nunes da Silva).

0925-2312/$ - see front matter © 2013 Elsevier B.V. All rights reserved.
http://dx.doi.org/10.1016/j.neucom.2013.07.015

travel, e.g., Dijkstra shortest path [9] and, for the same origin and the same destiny, the route is always the same. The Google Maps application, available in most Android operating systems, receive information about the traffic flow and try to calculate the route avoiding the roads with longer delays. However, even GPS routes being the shortest possible ones, if there is a lot of vehicles following the routes dictated by GPS, some of these routes will become quite crowded. Therefore, there is a good chance that the shortest route will not be the fastest one.

The works by Qu et al. [10] and Liu and Huang [11] define the shortest routes in relation to time and distance, respectively, using the model of urban traffic in Petri net. In the work by Qu et al., net transitions present an associated cost, which is calculated by analyzing images from urban roads. The total travel time is considered as the sum of such transition costs obtained during the corresponding Petri net simulation. In the work by Liu et al., places of the net correspond to edges of the graph and contain an associated time, which is equivalent to the distance to be traveled, and net transitions correspond to the vertices of the graph. Thus, simulation is done using the Petri net model, and the time that tokens remain in each place are added up until the final transition is fired. While computing the route for one vehicle the aforementioned works disregard other vehicles routes, so the shortest path found by them does not necessarily represent the fastest one. This can lead to congestion situations on some roads.

If alternate routes are given considering the possible congestion in the roads, even traveling a longer distance a vehicle may spend less time to arrive at its destination. However, depending on the size of the resulting Petri net, i.e., the size of urban traffic model to be considered during optimization, the complexity of the analysis can become too large, preventing its implementation in real time. Genetic Algorithms [8], in contrast, use the concept of schemas and their probabilistic interactions, based on the terminology of genetic biology, to find satisfactory solutions within a search space defined a priori.

Therefore, this work presents a new methodology that utilizes the analysis of Petri net as the fitness function of a Genetic Algorithm. The result of the fitness function is the total time that vehicles spend to reach their target destinations. To achieve this goal we developed an algorithm based on the analyses of High Level Petri net to offer a vehicle the best possible route with respect to travel time. Our main contribution is twofold: we take into consideration the routes of other vehicles in the traffic stream; and we optimize the routes in real time, i.e., considering information about traffic lights intervals as well as startup-lost time of vehicles.

The remainder of this paper is organized as follows. Section 2 presents an overview about urban traffic streams and the High Level Petri net model created to represent them. Section 3 shows the tests results and a discussion about them. Finally, Section 4 provides some conclusions.

2. Materials and methods

For each vehicle that starts its travel on a monitored urban area, the developed algorithm operates in defining its route, taking the current vehicle's position as the trip origin. The driver must inform the system the intended destination. After defining the possible routes of all vehicles in the urban traffic flow, the algorithm defines the best route with respect to travel time. As follows, in Section 2.1 we describe the Urban Traffic Petri net model developed; in Section 2.2 we explain how to compute both the travel time and the routes for a set of vehicles using the developed Petri net model; and in Section 2.3 we present the Genetic Algorithm used.

2.1. Petri net urban traffic model

Traffic streams are made up of individual drivers in their vehicles, the physical elements of the roadway, and the general environment. It is important to note that the interaction of drivers and corresponding vehicles with each other must be considered. In urban centers the traffic flow is interrupted, since it incorporates external intermissions into their operation such as, e.g., the time intervals in which the traffic lights remain red. These interruptions create queues when the signal is on red interval and, if there is no congestion, these queues dissipate during the green interval [12].

There are some works in the area of modeling and optimization of urban traffic using Petri nets. In the work by Di Febbraro et al. [13], an urban network of signalized intersection is modeled with a Hybrid Petri net, in which vehicle behavior is described by a time-oriented model and the traffic lights are represented dynamically by a discrete event model. From this model, the problem of coordinating multiple traffic lights in order to optimize the time taken by some special types of vehicles, such as public vehicles and emergency ones, is solved by dynamically changing the transitions firing time from a given route. Vàzquez et al. [14] present a Hybrid Petri net model of a single urban traffic intersection. Places of this model hold tokens to which continuous values are associated. These values represent the time that vehicles spend waiting in the traffic queues. Such values are used for the dynamic analysis of traffic behavior.

In our work High Level Petri nets are used to model the traffic stream of a controlled area of the city. In Fig. 1 it is shown that the complete model which contains 49 places representing the roads and 80 transitions is used to change a vehicle from one road to another. The vehicles are represented by tokens having as attributes route-vector and track-time as described in the following paragraphs.

The current position of each vehicle is taken as a starting point for IOPT Tools [15] to generate a state space. Possible routes for a vehicle are found from the corresponding state space analysis. Each of these routes is tested as the route-vector during optimization phase. This procedure is repeated for each vehicle that enters the network.

Another attribute of the token is associated with its time, which is a delay the system has to wait before the token becomes available to be consumed, i.e., it represents the time that the vehicle takes to go through a track. This attribute is named the track-time. It is important to note that even when the global clock reaches the appropriate time, the token will only be consumed if it is the first element of the queue of a place. This restriction ensures that the tokens are processed in the same order the vehicles they represent are traveling on the urban roads (First-In, First-Out policy).

The Petri net model used in this project consists in the High Level Petri net, which was chosen to allow the association of information to tokens, transitions and arcs. Examples of the information considered are data structures, variables, and intervals of time. The functions associated to transitions consist of guards that check if the next place on the vector-route associated with the token matches the output place of the transition. If they do not match, the transition is not fired.

2.2. Petri net analyses

Petri net can be represented by two matrices, one indicating the sets of places that are the input for each transition, and another indicating the sets of places that are the output for such transitions. The first one is the input matrix, also called backward incidence matrix. This matrix is described by $D^-_{(j,i)} = I(p_i, t_j)$, which means that each element of input matrix D^- corresponds to the number of arcs connecting place p_i to transition t_j. The second matrix is the output one, which is also called forward incidence

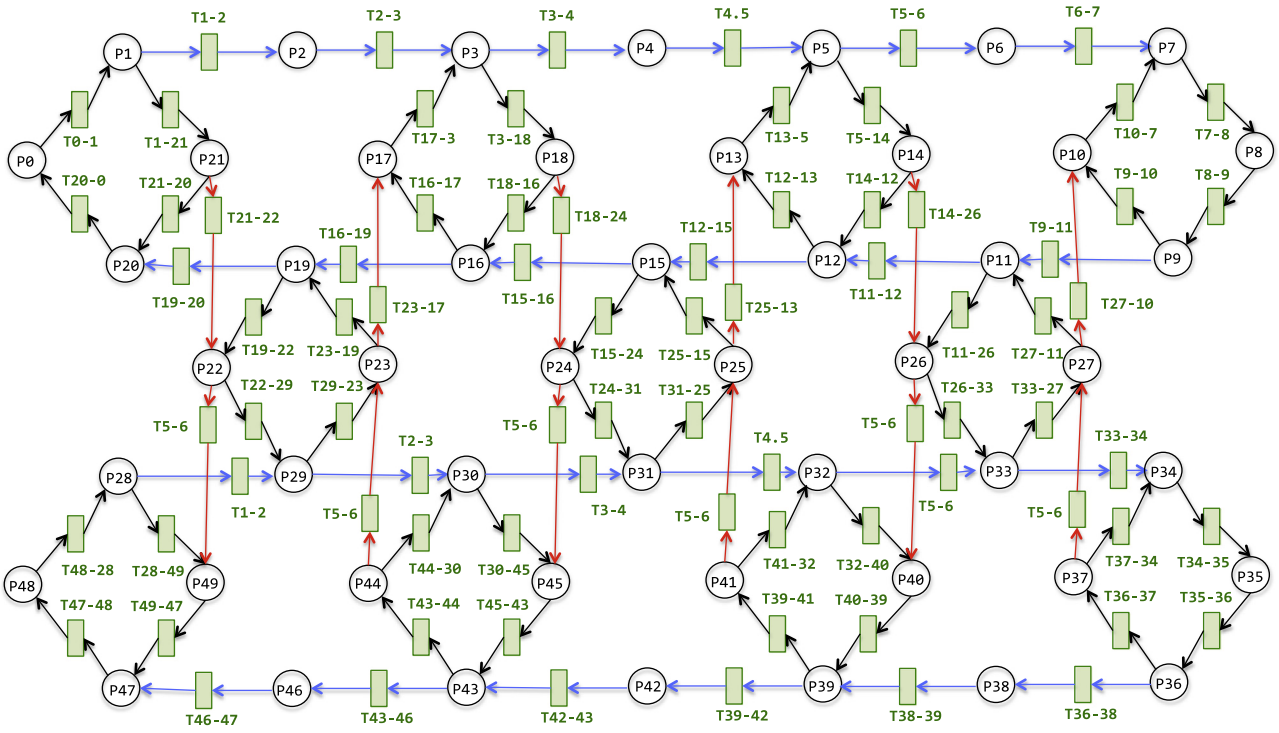

Fig. 1. Petri net model of the urban traffic. (For interpretation of the references to color in this figure caption, the reader is referred to the web version of this article.)

matrix. This matrix is described by $D^{+}_{(j,i)} = O(t_j, p_i)$, which means that each element of the output matrix D^{+} corresponds to the number of arcs connecting each transition t_j to a place p_i. The characteristic matrix D is obtained by subtracting the backward incidence matrix from the forward incidence matrix.

We call the firing vector of a transition a unit vector e[j] having one component corresponding to each transition of the Petri net. Firing of transition t_j is represented by vector e[j] where all but the jth component is zero. A transition t_j is enabled at any marking M, provided that the marking is greater than or equal to the firing vector of the transition in question multiplied by the backward incidence matrix of the net, i.e., $M \geq e[j] \cdot D^{-}$. Having used a High Level Petri net model, transition firings are also conditioned to the track-time and route-vector associated with the tokens.

Using the matrix approach presented in the former paragraphs, which is more comprehensively described by Peterson [16], we analyze the Petri net as follows. Once defined the possible routes for the vehicle, the program calculates, for each route, the state equations of the High Level Petri net. At this stage the program considers all tokens related to the position and routes of other vehicles plus the traffic lights time intervals.

The analysis is completed as soon as the token reaches its final destination. After completing this analysis, the program obtains the time taken by the token to move through the predefined places on the route. The marking corresponding to the shortest time to reach the end marking is added to the net. Thus, the next vehicle arriving at the net will have the routes of other vehicles already calculated and the program will be able to find, in real time, the routes that have a lower amount of vehicles during the time over its trajectory.

2.3. Genetic Algorithm

The Genetic Algorithms are search algorithms based on mechanisms of natural selection. They combine the data structure of the fittest individuals using a random exchange of information. Their main goal is to strike a balance between efficiency and effectiveness required for use in different environments. Therefore, Genetic Algorithms are developed theoretically or empirically to provide robust search in complex space of solutions [17], such as scheduling systems with a large number of parameters, or in our case, defining routes of a lot of possibilities for incoming vehicles.

The chromosome of our Genetic Algorithm consists of an array of size equal to the number of vehicles in urban traffic network at an instant t. Each gene corresponds to one of the many routes of the given vehicle. Thus, during the selection of individuals and their genetic operations, the positions of the genes are not altered so as to maintain the order of the respective vehicles.

Initially a population is randomly generated from the route-vectors produced by the Petri net for each vehicle. Then, the fitness of each individual is computed according to Eq. (1); where $V(i,r)$ refers to the total time spent by vehicle i for traversing the route r, i.e., the value of each gene in the chromosome under evaluation, and N is the amount of vehicles at instant t. Thus, the fitness function corresponds to the simulation of the Petri net for a given set of routes

$$fitness = \min \sum_{i=0, r=0}^{N} [V_{(i,r)}] \tag{1}$$

After computing the fitness for each individual in the population we use the roulette wheel selection to choose the individuals from the population to apply crossover. The parents utilized for the crossover operation are discarded and the offsprings are added to a new population. Once the new generation is completed the process is repeated. This occurs until either all population converges to a winning individual or a time limit of 2 s is reached. The time limit is related to the maximum time within which the system must return a response to the traffic controller, and depends on the vehicles saturation on the roads.

3. Results and discussion

To validate the proposed methodology we developed tests based on the model presented in Fig. 1. The results obtained from

such tests are compared to the corresponding results from Dijkstra's Shortest Path algorithm to demonstrate the efficiency of the methodology.

In this scenario it is assumed that a vehicle starts its journey in place *P0* towards place *P34* every 2 s. From the definition of the routes using Dijkstra's algorithm, assume that the shortest path to be followed consists, in order, of places $R_D = \{P0, P1, P2, P3, P4, P5, P14, P26, P33, P34\}$. The amount of vehicles in each place of the network over time is presented in Fig. 2. As can be seen the maximum time required for a vehicle to arrive at its destination is 336 s.

Observe that most of the routes used keep the maximum amount of vehicles for a certain time, since the Dijkstra's algorithm determines the same route for all vehicles. In Fig. 1, the yellow line represents the amount of vehicles in place *P26*, which precedes the final destination (*P34*). Taking into consideration a block size of 100 m we empirically determined a maximum amount of 10 vehicles per block. As displayed in Fig. 1, from time $t = 130$ s to $t = 330$ s place *P26* is at its full capacity, meaning that it holds the maximum allowed number of tokens. This situation results in a long waiting time for vehicles as well as increases the probability of bottlenecks if any incident occurs in this path.

The graph in Fig. 3 shows the amount of tokens in each and every place throughout their paths in the network with respect to time as computed by the proposed algorithm. This test differs from the former one by the fact that each vehicle has up to four possible routes, each one defined from the analysis of the state space of the Petri net. The Genetic Algorithm operates in defining the best set of routes to optimize travel time both individually and globally. Comparing Fig. 3 to Fig. 2 one can see that the total travel time decreases from 336 s to 318 s. Furthermore, there is a decrement on the total amount of vehicles in all ways and on the use of pathways not previously considered by Dijkstra's algorithm. Moreover, the yellow line shows that even place *P26* sporadically

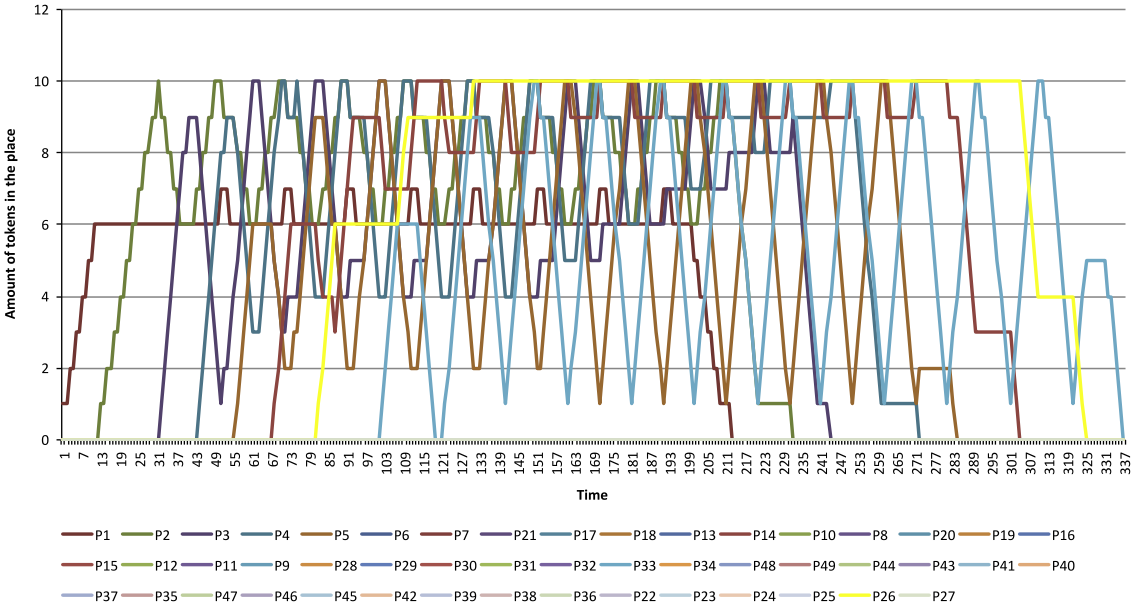

Fig. 2. Quantity of vehicles over time using Dijkstra's Algorithm.

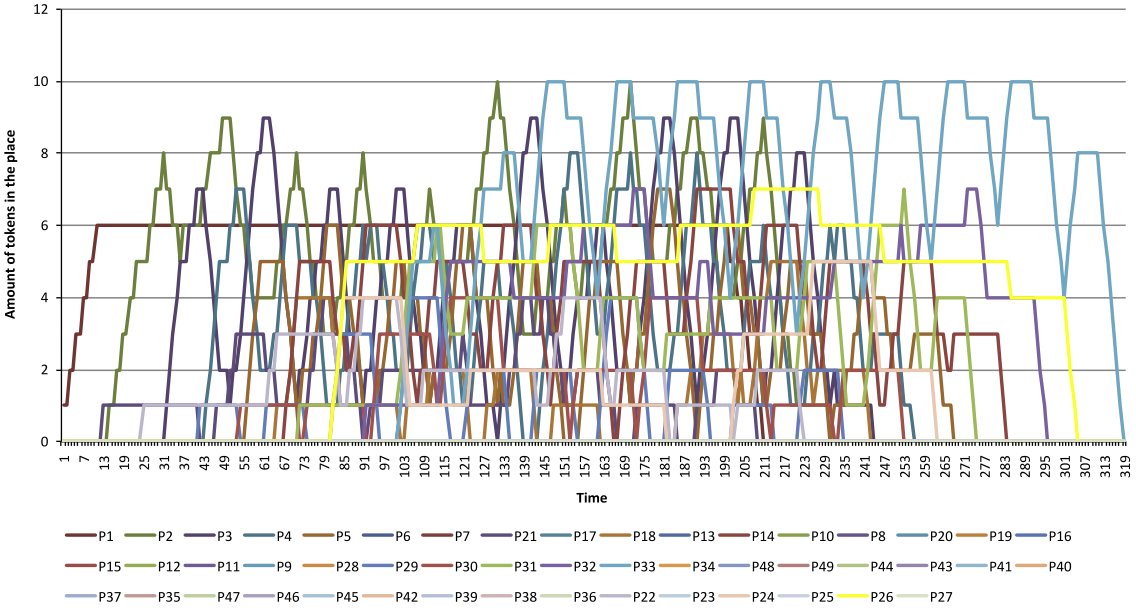

Fig. 3. Amount of vehicles (with four routes) over time using the Genetic Algorithm.

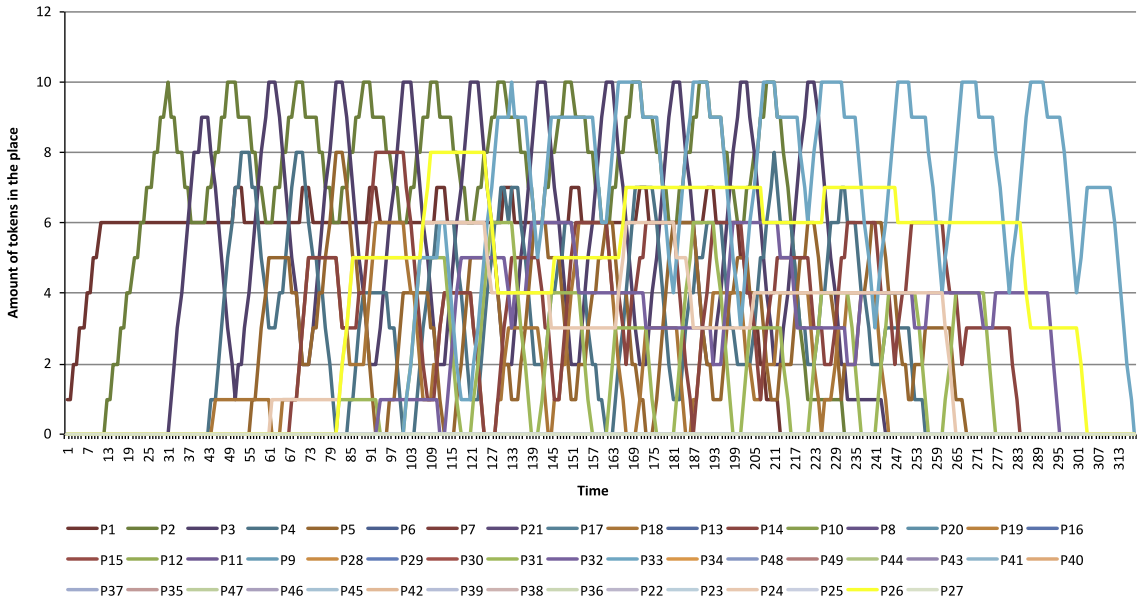

Fig. 4. Amount of vehicles (with two routes) over time using the Genetic Algorithm.

keeping a constant amount of vehicles for a few seconds, it does not reach its maximum capability.

The graph in Fig. 4 shows the amount of vehicles over time considering two routes per vehicle. It is verified that there was still an optimization in the total time taken by vehicles and a decrease in the amount of vehicles per lane. However, as fewer paths are considered, this solution uses Dijkstra's Shortest Path algorithm and hence the problems previously reported even with a lower probability can occur.

The Genetic Algorithms utilized in these tests had a population of 2000 individuals and 20 generations, differing only by the amount of available routes for each vehicle. Increasing the population size or the number of generations did not influence the solution provided by the algorithm. However, the decrease in population size, in some cases, returned unsuccessful solutions when the vehicles were associated to a greater number of routes.

Besides achieving more than 5% reduction in the total time required to dispatch all vehicles, the approach proposed in this paper indicates a significant improvement in urban traffic, since it distributes vehicles for roads not considered by Dijkstra's algorithm.

4. Conclusion

It is concluded that the coordinated distribution used in the definition of the routes ensures a shorter route for all drivers. The use of Genetic Algorithm proved efficient in the search for solutions in a timely manner, compatible to real time urban traffic requirements. It has also been found that the more routes are considered, the greater vehicle delivery for urban roads with equivalent travel times are achieved. When compared with Dijkstra's Shortest Path algorithm, there was considerable time savings to dispatch all vehicles and also not to overload pathways.

References

[1] J. Vales-Alonso, F. Vicente-Carrasco, J.J. Alcaraz, Optimal configuration of roadside beacons in V2I communications, Computer Networks 55 (2011) 3142–3153.
[2] L. Du, S. Ukkusuri, W.F.Y. Del Valle, S. Kalyanaraman, Optimization models to characterize the broadcast capacity of vehicular ad hoc networks, Transportation Research. Part C: Emerging Technologies 17 (2009) 571–585.
[3] A. Zhang, Z. Gao, Effect of ATIS information under incident-based congestion propagation, Procedia – Social and Behavioral Sciences 43 (2012) 628–637.
[4] P. Kumar, V. Singh, D. Reddy, Advanced traveler information system for Hyderabad city, IEEE Transaction on Intelligent Transportation Systems 6 (2005) 26–37.
[5] D. Ni, Determining traffic-flow characteristics by definition for application in ITS, IEEE Transaction on Intelligent Transportation Systems 8 (2007) 181–187.
[6] T. Murata, Petri nets: properties, analysis and applications, Proceedings of the IEEE 77 (1989) 541–580.
[7] F. Pereira, F. Moutinho, L. Gomes, J. Ribeiro, R. Campos-Rebelo, An IOPT-net state-space generator tool, in: Proceeding of the 9th IEEE International Conference on Industrial Informatics (INDIN 2011), Lisbon, Portugal, 2011, pp. 383–389.
[8] D.E. Goldberg, Genetic Algorithms in Search, Optimization and Machine Learning, Addison-Wesley, 1989.
[9] X. Wang, H. Qu, Z. Yi, A modified pulse coupled neural network for shortest-path problem, Neurocomputing 72 (2009) 3028–3033.
[10] Y. Qu, L. Li, Y. Liu, Y. Chen and Y. Dai, Travel routes estimation in transportation systems modeled by Petri nets, in: IEEE International Conference on Vehicular Electronics and Safety, 2010, pp. 73–77.
[11] X. Liu, S. Huang, The simulation algorithm of military transportation shortest path based on Petri net, in: ISECS International Colloquium on Computing, Communication, Control, and Management 2009, pp. 111–114.
[12] P. Roess, E.E. Prassas, W.E. MacShane, Traffic Engineering, fourth ed., Prentice Hall, 2011.
[13] A. Di Febbraro, D. Giglio, N. Sacco, Urban traffic control structure based on hybrid Petri nets, IEEE Transactions on Intelligent Transportation Systems 5 (2004) 224–237.
[14] C.R. Vàzquez, H.Y. Sutarto, R. Boel, M. Silva, Hybrid Petri net model of a traffic intersection in an urban network, in: IEEE International Conference on Control Applications, Yokohama, 2010, pp. 658–664.
[15] L. Gomes, J.P. Barros, A. Costa, R. Nunes, The input–output place-transition Petri net class and associated tools, in: IEEE International Conference on Industrial Informatics, 2007, Vienna, pp. 509–514.
[16] J.L. Peterson, Petri Net Theory and the Modeling of Systems, Prentice Hall, 1981.
[17] J. Holland, Adaptation in Natural and Artificial Systems, University of Michigan Press, 1975.

Henrique Dezani received the B.S. degree in Computer Science, from Rio Preto University Center, Brazil, in 2003. He received his M.Sc. from São Paulo State University, Brazil, in 2006, and his Ph.D. in Electrical Engineering from the University of Campinas, Brazil, in 2012. In 2006 he joined Faculty of Technology of Rio Preto, at São José do Rio Preto, Brazil, where he is an Associate Professor. During February–July of 2012, he has been conducting part of his Ph.D. work at the Universidade Nova de Lisboa, Portugal. His research interests include the modeling, analysis, development and optimization of digital systems, embedded systems, intelligent systems and remote sensing.

Regiane D.S. Bassi received a degree in Mathematics from the Dom Bosco School of Arts, Sciences and Education, Monte Aprazível, Brazil, in 2006. She also has a degree of Specialist in Teaching for Primary, Secondary and Higher Education, from the Northern São Paulo University Center, in 2009. She is now working towards her M.Sc. degree in Computer Science at the University of São Paulo, São Carlos, Brazil. Her research experience includes mathematics and physics education.

Norian Marranghello, received the Electronic Engineering degree from Pontifícia Universidade Católica do Rio Grande do Sul, Brazil, in 1982. He received his M.Sc. and Ph.D. degrees, both in Electrical Engineering, from the University of Campinas, Brazil, respectively, in 1987 and 1992. During 1997–1998, he has been a postdoctoral fellow at the University of Aarhus, Denmark. In 1998 he received a Licentiate degree in Digital Systems from São Paulo State University. In 1992 he joined São Paulo State University, at Rio Preto, Brazil, where he is now Full Professor of Computer Science. His research interests include digital systems design and analysis, reconfigurable hardware, and intelligent systems.

Luís Gomes received his Electrotechnical Engineering degree from Universidade Técnica de Lisboa, Portugal, in 1981, and a Ph.D. degree in Digital Systems from Universidade Nova de Lisboa (UNL), in 1997. He is an Associate Professor at the Electrical Engineering Department, Faculty of Sciences and Technology, UNL, Portugal, and a researcher at UNINOVA Institute, Portugal. From 1984 to 1987, he was with EID, a Portuguese medium-sized enterprise, in the area of electronic system design, in the RD engineering department. His main interests include the usage of Petri nets and other concurrency models applied to reconfigurable and embedded systems co-design.

Furio Damiani received the Electrical Engineering degree from the University of São Paulo, Brazil, in 1969. He received his M.Sc. and Ph.D. degrees, both in Electrical Engineering, from the University of Campinas, Brazil, respectively, in 1976 and 1982. In 1974 he joined the University of Campinas, Brazil, where he is an Associate Professor. His research interests include digital systems design and reconfigurable hardware.

Ivan Nunes da Silva received a B.S. degree in Computer Science and an Electrical Engineering degree, both from Federal University at Uberlandia, MG, Brazil, respectively, in 1991 and 1992. He received M.Sc. and Ph.D. degrees, both in Electrical Engineering, from the University of Campinas, Brazil, respectively, in 1995 and 1997. He is now an Associate Professor at the Department of Electrical Engineering, of the University of São Paulo, at São Carlos, Brazil. He has been a researcher of the CNPq since 2000. His research areas are related to intelligent automation, including electric power systems, intelligent control of machines and equipment, design of intelligent systems architecture, and identification and systems optimization.